

WIPRO CAPSTONE PROJECT MODULE 6

IVI Media Service and Audio HAL Message Flow Simulation

Submitted By

Parthib Ranjan Sarbajna

Enrollment Number: 12023002028150

Registration Number: 304202300901068

Institute of Engineering and Management, Kolkata

B.Tech in CSE (AI & ML)

Project Report

Abstract

In-Vehicle Infotainment (IVI) systems provide a software platform through which vehicle occupants interact with media, audio, navigation and other vehicle-related information services. A media playback request in an IVI system does not directly control the vehicle speakers. Instead, the request passes through several software layers before reaching the lower-level audio hardware.

This project presents a simplified simulation of message flow between the Media Service and Audio Hardware Abstraction Layer (Audio HAL) within an IVI audio system. The implementation was developed using Python and executed using Google Colab. The simulation represents a simplified stack consisting of the User, IVI User Interface, Media Service, Audio Framework, Audio HAL, Audio Driver and Vehicle Speakers.

Five common user operations were simulated: Play, Pause, Resume, Volume Change and Stop. Each operation generates timestamped Android-style system logs showing how the command propagates through the different layers of the system. A total of 30 system messages are generated during the complete simulation.

The project also produces an architecture diagram and statistical analysis of messages processed by the individual components. The objective is not to implement a production Android Automotive audio stack, but to provide a clear conceptual and executable demonstration of how user-level media commands propagate through an IVI software architecture.

Keywords: IVI, In-Vehicle Infotainment, Media Service, Audio HAL, Audio Framework, Audio Driver, Android Automotive, Message Flow, Python, Google Colab.

Contents

Abstract	1
1 Introduction	7
1.1 Background	7
1.2 In-Vehicle Infotainment Systems	7
1.3 Problem Statement	8
1.4 Project Objectives	8
1.5 Project Scope	9
2 Technical Background	10
2.1 IVI Architecture	10
2.2 Media Service	10
2.3 Audio Framework	11
2.4 Audio Hardware Abstraction Layer	11
2.5 Audio Driver	11
2.6 Vehicle Speakers	12
3 System Architecture and Message Flow	13
3.1 Overall Architecture	13
3.2 Architecture Diagram	13
3.3 Message Propagation	14
3.4 Command Flow	14
4 Methodology	16

4.1	Development Environment	16
4.2	Simulation Workflow	16
4.3	Logging Mechanism	17
4.4	Simulated Operations	17
4.4.1	Play	17
4.4.2	Pause	18
4.4.3	Resume	18
4.4.4	Volume Change	18
4.4.5	Stop	18
5	Implementation	19
5.1	Software Components	19
5.2	Architecture Graph	19
5.3	Log Generation	20
5.4	Data Analysis	20
6	Results and Analysis	21
6.1	Simulation Result	21
6.2	Message Statistics	21
6.3	Message Processing Visualization	21
6.4	Generated System Logs	22
6.5	Event Propagation Analysis	23
7	Discussion	24
7.1	Importance of Layered Communication	24
7.2	Role of the Audio HAL	24
7.3	Importance of System Logs	25
7.4	Educational Value	25
8	Limitations	26
8.1	Simplified Architecture	26

8.2	No Physical Audio Hardware	26
8.3	No Android Automotive Emulator	26
8.4	Simulated Logs	26
8.5	Simplified Audio Processing	27
9	Future Scope	28
9.1	Android Automotive Integration	28
9.2	Real Android Logs	28
9.3	Audio Focus Simulation	28
9.4	Navigation and Media Interaction	28
9.5	Real-Time Dashboard	29
9.6	Hardware Deployment	29
10	Conclusion	30
A	Project Repository	32
A.1	Repository Structure	32
B	Simulation Log Structure	33

List of Figures

3.1	IVI Media Service to Audio HAL Message Flow	14
6.1	Message Processing by IVI System Component	22

List of Tables

4.1	Development Environment	16
5.1	Software Libraries Used	19
6.1	Simulated IVI Operations	21
6.2	Simulation Summary	21
B.1	Log Entry Fields	33

Chapter 1 Introduction

1.1 Background

Modern vehicles contain software-intensive systems that provide occupants with media playback, navigation, communication and information services. An important part of this software environment is the In-Vehicle Infotainment (IVI) system.

An IVI system provides the interface through which a user can interact with audio and media services. Although the user may see only a simple Play, Pause or Volume button, the resulting operation can involve multiple software components.

A simplified representation of the audio command path used in this project is:

$$User \rightarrow IVI\ UI \rightarrow Media\ Service \rightarrow Audio\ Framework \rightarrow Audio\ HAL \rightarrow Audio\ Driver \rightarrow Speaker \quad (1.1)$$

The purpose of this project is to represent this communication path through a software simulation.

1.2 In-Vehicle Infotainment Systems

An IVI system acts as a central interaction platform inside a vehicle. Depending on the vehicle architecture, it can provide features such as:

- Music and media playback.
- Audio volume control.
- Navigation.
- Bluetooth and phone connectivity.
- Voice interaction.

- Vehicle information.
- Communication services.

For this project, the focus is specifically on the media and audio portion of the IVI architecture.

1.3 Problem Statement

When a user performs a media operation such as pressing the Play button, the command must be processed by multiple software components before audio can be produced by the vehicle speakers.

Understanding this propagation is important for studying IVI software architecture and debugging communication between different layers.

The problem addressed by this project is therefore:

To simulate and visualize how a user-generated media command propagates through the Media Service, Audio Framework, Audio HAL and lower-level audio components in an IVI system.

1.4 Project Objectives

The major objectives of the project are:

1. To understand the basic architecture of an IVI audio system.
2. To model the interaction between the Media Service and Audio HAL.
3. To simulate user-generated media commands.
4. To generate Android-style timestamped system logs.
5. To visualize the communication path between system components.
6. To analyze the number of messages processed by each component.
7. To demonstrate the propagation of Play, Pause, Resume, Volume and Stop operations.

1.5 Project Scope

The project is intentionally designed as a simplified educational simulation. It does not attempt to reproduce the complete Android Automotive Operating System audio stack.

The scope includes:

- User input simulation.
- Media command propagation.
- Audio framework interaction.
- Audio HAL interaction.
- Audio driver representation.
- Simulated speaker output.
- Log generation.
- Message statistics.

The project does not include a real Android Automotive emulator, physical vehicle audio hardware or a production Audio HAL implementation.

Chapter 2 Technical Background

2.1 IVI Architecture

The In-Vehicle Infotainment architecture can be viewed as a collection of software and hardware layers. The upper layers interact with the user, while lower layers communicate with hardware.

The simplified architecture considered in this project consists of:

1. User
2. IVI User Interface
3. Media Service
4. Audio Framework
5. Audio HAL
6. Audio Driver
7. Vehicle Speakers

2.2 Media Service

The Media Service is responsible for processing media-related commands generated by the IVI user interface.

Examples of media commands include:

- PLAY
- PAUSE
- RESUME

- STOP
- VOLUME CHANGE

In the simulation, the Media Service receives the command from the IVI interface and forwards an appropriate request toward the audio subsystem.

2.3 Audio Framework

The Audio Framework represents the software layer responsible for managing audio-related requests before they reach the hardware abstraction layer.

For example, when a Play command is received, the Audio Framework creates an audio playback request and forwards it to the Audio HAL.

2.4 Audio Hardware Abstraction Layer

The Audio Hardware Abstraction Layer, commonly referred to as Audio HAL, provides an abstraction between the higher-level audio framework and the lower-level hardware implementation.

In the simplified simulation, the Audio HAL performs operations such as:

- Opening the audio output stream.
- Initializing audio output.
- Pausing playback.
- Resuming playback.
- Changing hardware volume.
- Closing the audio output stream.

2.5 Audio Driver

The Audio Driver represents the lower-level software responsible for communicating with the audio hardware.

In the simulation, PCM audio data is represented as being transferred from the Audio HAL toward the audio driver.

2.6 Vehicle Speakers

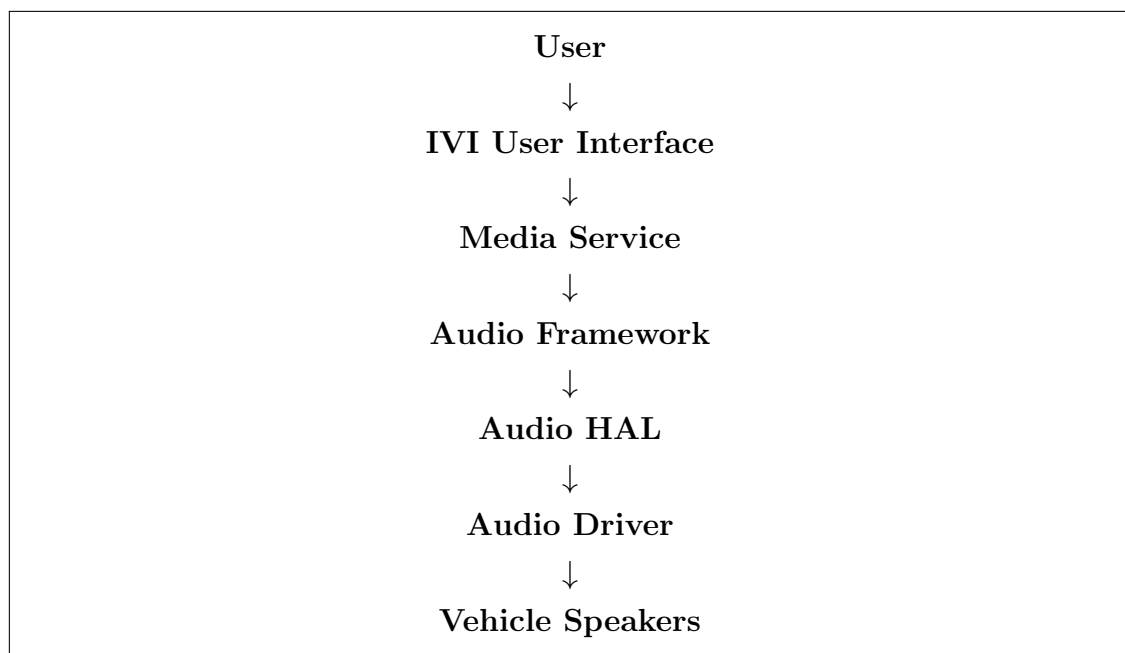
Vehicle speakers represent the final output stage of the simulated audio pipeline.

Once the lower-level audio components have processed the command, the system represents the corresponding audio state at the speaker output.

Chapter 3 System Architecture and Message Flow

3.1 Overall Architecture

The complete message flow implemented in the project is:



3.2 Architecture Diagram

The Python implementation automatically generates the architecture diagram showing the communication path between the major IVI components.

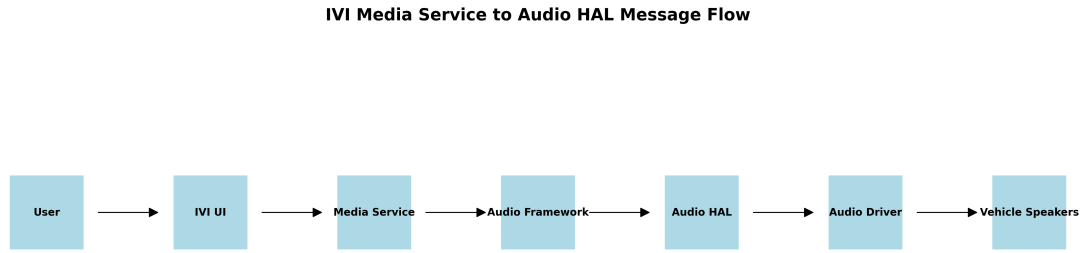


Figure 3.1: IVI Media Service to Audio HAL Message Flow

The diagram illustrates the directional flow of commands from the user interface toward the final audio output.

3.3 Message Propagation

The message propagation can be summarized as follows:

1. The user performs an operation through the IVI interface.
2. The IVI interface generates the corresponding command.
3. The Media Service receives the command.
4. The Media Service communicates with the Audio Framework.
5. The Audio Framework generates an audio request.
6. The Audio HAL processes the hardware-level audio request.
7. The Audio Driver represents communication with the audio hardware.
8. The vehicle speakers represent the final audio output.

3.4 Command Flow

The general command flow is:

$$User\ Input \rightarrow Media\ Command \rightarrow Audio\ Request \rightarrow HAL\ Operation \rightarrow Audio\ Output \quad (3.1)$$

This provides a simplified model of the interaction between high-level user actions and lower-level audio components.

Chapter 4 Methodology

4.1 Development Environment

The project was implemented using Python in Google Colab. The environment provides an interactive notebook platform suitable for executing the simulation and visualizing the generated results.

Table 4.1: Development Environment

Component	Value
Platform	Google Colab
Programming Language	Python
Data Processing	Pandas
Graph Visualization	NetworkX
Plotting	Matplotlib
Output Format	Text Logs and PNG Images

4.2 Simulation Workflow

The complete workflow consists of the following stages:

1. Define the IVI system components.
2. Construct the message-flow graph.
3. Implement a logging function.
4. Simulate user operations.
5. Record component-level events.
6. Store the generated logs in a Pandas DataFrame.
7. Generate message statistics.

8. Generate visualization outputs.
9. Save the final logs and figures.

4.3 Logging Mechanism

A custom Python logging function was implemented to record:

- Timestamp.
- Component name.
- Event type.
- Message description.

Each generated log entry follows the general format:

Timestamp Component Event Message

For example:

01:32:10.101 IVI UI USER_INPUT User pressed PLAY button
01:32:10.105 Media Service COMMAND_RX PLAY command received
01:32:10.112 Audio Framework AUDIO_REQUEST
Creating audio playback request
01:32:10.118 Audio HAL HAL_OPEN
Opening audio output stream

The exact timestamps vary depending on the execution time of the simulation.

4.4 Simulated Operations

Five operations were implemented.

4.4.1 Play

The Play operation represents the initiation of audio playback.

The sequence is:

PLAY $\rightarrow$ *Media Service* $\rightarrow$ *Audio Framework* $\rightarrow$ *Audio HAL* $\rightarrow$ *Driver* $\rightarrow$ *Speakers*
(4.1)

4.4.2 Pause

The Pause operation stops the current audio output while maintaining the simulated playback state.

4.4.3 Resume

The Resume operation restarts the audio output after a pause command.

4.4.4 Volume Change

The Volume Change operation demonstrates propagation of a volume command through the Media Service, Audio Framework and Audio HAL.

4.4.5 Stop

The Stop operation terminates playback and closes the simulated audio output stream.

Chapter 5 Implementation

5.1 Software Components

The implementation uses three primary Python libraries.

Table 5.1: Software Libraries Used

Library	Purpose
Pandas	Log storage and statistical analysis
NetworkX	IVI architecture graph generation
Matplotlib	Visualization of message statistics

5.2 Architecture Graph

NetworkX was used to represent the IVI architecture as a directed graph.

The graph contains seven primary nodes:

- User
- IVI UI
- Media Service
- Audio Framework
- Audio HAL
- Audio Driver
- Vehicle Speakers

The edges represent the direction of message propagation.

5.3 Log Generation

For each system event, the simulation creates a structured record:

```
{
  "timestamp": timestamp,
  "component": component,
  "event": event,
  "message": message
}
```

These records are subsequently converted into a Pandas DataFrame.

5.4 Data Analysis

After the simulation completes, the generated log data is analyzed to determine:

- Total number of messages.
- Number of components involved.
- Number of event types.
- Number of messages processed by each component.
- Frequency of each event type.

Chapter 6 Results and Analysis

6.1 Simulation Result

The complete simulation successfully executed all five defined operations:

Table 6.1: Simulated IVI Operations

Operation	Purpose	Result
Play	Start audio playback	Playback started
Pause	Temporarily stop playback	Playback paused
Resume	Continue playback	Playback resumed
Volume Change	Modify output volume	Volume updated
Stop	Terminate playback	Playback stopped

6.2 Message Statistics

The complete simulation generated a total of 30 system messages.

Seven components participated in the simulated communication flow.

Table 6.2: Simulation Summary

Parameter	Value
Total operations	5
Total messages	30
System components	7

6.3 Message Processing Visualization

The implementation generates a bar chart showing the number of messages associated with each system component.

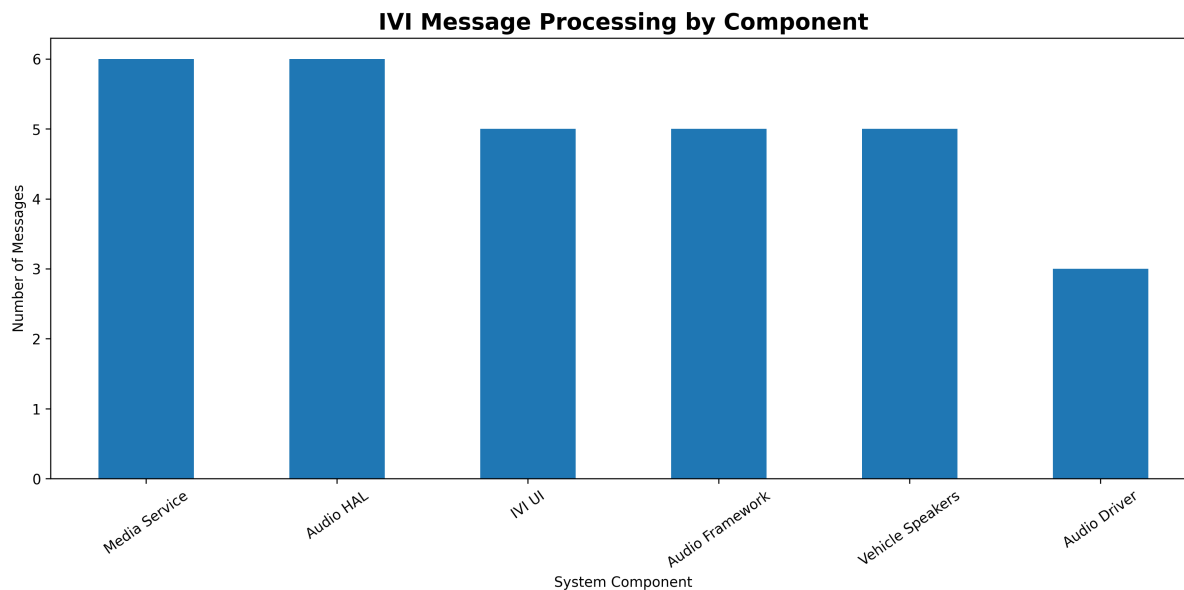


Figure 6.1: Message Processing by IVI System Component

The visualization provides a simple way to understand how many simulated events are associated with the different layers of the IVI system.

6.4 Generated System Logs

The project generates a text file named:

```
simulated_android_logs.txt
```

The file contains the timestamp, component, event and message associated with every simulated operation.

A representative portion of the generated log is shown below:

```
IVI UI | User pressed PLAY button
Media Service | PLAY command received from IVI UI
Media Service | Preparing selected media track
Audio Framework | Creating audio playback request
Audio HAL | Opening audio output stream
Audio HAL | Audio output stream initialized
Audio Driver | Receiving PCM audio data
Vehicle Speakers | Audio output started
```

6.5 Event Propagation Analysis

The simulation demonstrates that a user action is not directly translated into speaker output. Instead, the command passes through multiple intermediate layers.

For example, a Play request follows:

$$User \rightarrow IVI UI \rightarrow Media Service \rightarrow Audio Framework \rightarrow Audio HAL \rightarrow Audio Driver \rightarrow Speaker \quad (6.1)$$

This layered architecture provides separation between user-level media logic and lower-level hardware interaction.

Chapter 7 Discussion

7.1 Importance of Layered Communication

The simulation demonstrates the importance of separating responsibilities within an IVI audio architecture.

The IVI User Interface is responsible for receiving user input, while the Media Service handles media-related commands. The Audio Framework manages audio requests, and the Audio HAL provides an abstraction toward lower-level audio components.

Such separation allows individual layers to perform specialized functions.

7.2 Role of the Audio HAL

The Audio HAL represents an important boundary between the software audio framework and hardware-specific implementation.

In the simulation, the Audio HAL handles operations including:

- Opening an audio stream.
- Pausing an audio stream.
- Resuming an audio stream.
- Configuring volume.
- Closing an audio stream.

The simulation therefore provides a simplified representation of why an abstraction layer is useful in an automotive software environment.

7.3 Importance of System Logs

System logs are useful for understanding the sequence of operations in a multi-layer software system.

For an IVI system, logs can help developers identify:

- Whether a user command was received.
- Whether the Media Service processed the command.
- Whether an audio request was generated.
- Whether the Audio HAL received the request.
- Whether audio output was initiated or stopped.

The project demonstrates this concept using simulated Android-style logs.

7.4 Educational Value

The project provides an executable representation of an IVI audio architecture rather than only a static flowchart.

The combination of:

- Architecture visualization.
- Simulated commands.
- Timestamped logs.
- Statistical analysis.

makes it possible to observe the complete command propagation process.

Chapter 8 Limitations

8.1 Simplified Architecture

The implemented architecture is a simplified representation of an IVI audio system. A real Android Automotive system contains additional services, interfaces, processes and hardware-specific components.

8.2 No Physical Audio Hardware

The project does not communicate with actual vehicle speakers or an actual automotive Audio HAL implementation. The speaker and driver components are represented through simulation.

8.3 No Android Automotive Emulator

The implementation was executed as a Python simulation in Google Colab. Therefore, it does not demonstrate direct deployment inside an Android Automotive operating environment.

8.4 Simulated Logs

The generated logs follow an Android-style structure but are produced by the Python simulation. They should not be interpreted as logs captured from a physical Android Automotive device.

8.5 Simplified Audio Processing

Actual audio systems involve detailed concepts such as PCM formats, sampling rates, buffers, audio focus, mixer behavior, routing and hardware-specific interfaces. These aspects are outside the scope of the current project.

Chapter 9 Future Scope

The project can be extended in several directions.

9.1 Android Automotive Integration

The simulation can be extended into a basic Android Automotive application that communicates with actual Android media services.

9.2 Real Android Logs

Future work can capture real Android Automotive logs using tools such as ADB and analyze the communication between media-related services and the audio subsystem.

9.3 Audio Focus Simulation

The project can be extended to simulate audio focus events. For example, an incoming phone call could interrupt media playback and allow playback to resume after the call ends.

9.4 Navigation and Media Interaction

The IVI architecture can be extended to include navigation and projection services alongside media playback.

A larger architecture could be represented as:

$$User \rightarrow IVI UI \rightarrow \begin{cases} Media Service \\ Navigation Service \\ Projection Service \end{cases} \quad (9.1)$$

9.5 Real-Time Dashboard

A future version could provide a graphical dashboard showing live message propagation, component states and audio events.

9.6 Hardware Deployment

The simulation could eventually be connected to an embedded development platform or automotive hardware environment to demonstrate actual audio output and hardware-level interaction.

Chapter 10 Conclusion

This project presented a simplified simulation of message flow between the Media Service and Audio Hardware Abstraction Layer within an In-Vehicle Infotainment system.

The implementation was developed in Python and executed using Google Colab. The system models seven major components: User, IVI User Interface, Media Service, Audio Framework, Audio HAL, Audio Driver and Vehicle Speakers.

Five common media operations were simulated: Play, Pause, Resume, Volume Change and Stop. The complete simulation generated 30 system messages representing the propagation of commands through the IVI audio stack.

The project also generated an architecture diagram, Android-style timestamped logs and message-processing statistics. These outputs demonstrate how a user-level action passes through multiple software layers before reaching the simulated audio output.

The implementation is intentionally a conceptual and educational simulation rather than a production Android Automotive implementation. Nevertheless, it provides a practical understanding of the relationship between media services, audio framework components, the Audio HAL and lower-level audio components.

The project can be extended in the future through Android Automotive integration, real Android log collection, audio focus handling, navigation integration and hardware-based testing.

Bibliography

- [1] Android Developers, *Audio and Media Documentation*, Google Android Documentation.
- [2] Android Developers, *Android Automotive OS Documentation*, Google Android Documentation.
- [3] Python Software Foundation, *Python Documentation*.
- [4] Pandas Development Team, *Pandas Documentation*.
- [5] NetworkX Developers, *NetworkX Documentation*.
- [6] Matplotlib Development Team, *Matplotlib Documentation*.

Chapter A Project Repository

The complete implementation of the project is maintained in the GitHub repository:

<https://github.com/ParthibSarbjana/IVI-Media-AudioHAL-Simulation>

The repository contains:

- Google Colab Jupyter Notebook.
- README documentation.
- Python requirements file.
- IVI architecture diagram.
- Message statistics visualization.
- Simulated Android-style system logs.

A.1 Repository Structure

```
IVI-Media-AudioHAL-Simulation/  
|  
|-- IVI_Media_AudioHAL_Simulation.ipynb  
|-- README.md  
|-- requirements.txt  
|-- IVI_Message_Flow.png  
|-- IVI_Message_Statistics.png  
|-- simulated_android_logs.txt
```

Chapter B Simulation Log Structure

Each generated log entry contains four primary fields:

Table B.1: Log Entry Fields

Field	Description
Timestamp	Time at which the event was generated
Component	IVI component generating or receiving the event
Event	Type of system event
Message	Human-readable description of the operation

A simplified example is:

Timestamp	Component	Event
01:32:10.101	IVI UI	USER_INPUT
Message:		
User pressed PLAY button		

This format makes the simulated communication sequence easy to inspect and analyze.